API Test Case Documentation

Project: CS Blood Donation Drive Coordinator

API: Express REST API

Base path: `/api`

Document status: Test plan derived from the current server routes, controllers, and `server/test_sprints.js`.

1. Purpose and coverage labels

This document describes API checks for the system's health, authentication, drive scheduling, donor appointments, staff operations, analytics, notifications, and administration.

- **Existing automated assertion** means `server/test_sprints.js` contains an assertion for the stated behavior. It does not imply that the test was executed as part of producing this document.
- **Recommended** means the route or behavior exists in the code, but the current integration script does not assert the case.
- Expected outcomes below follow the current route/controller behavior. Where a controller delegates failure handling to the data store or centralized error handler, confirm the precise error response against that implementation before treating it as a stable API contract.

2. Execution setup

1. Configure the server environment using `server/.env.example`; supply a test database through `DATABASE_URL` and a non-production `JWT_SECRET`. Never use production donor data.

2. Start the backend integration suite from the repository:

```
cd server
node test_sprints.js
```

3. The integration script binds its test requests to `http://127.0.0.1:5099/api`. It uses seeded demo accounts and creates synthetic drive/donor data. Keep the database isolated from production and reset or clean up test records as appropriate.

4. For manual requests, use `http://localhost:5000/api` by default, unless `PORT` is configured otherwise. Supply a bearer token to protected endpoints:

```
Authorization: Bearer <test-jwt>
Content-Type: application/json
```

5. Use unique synthetic email addresses when registering users. Do not put real passwords, JWT signing secrets, or donor health answers into reports or source control.

3. Existing automated integration assertions

These cases are asserted by `server/test_sprints.js`.

API-001 — Health endpoint responds

- **Request:** `GET /api/health`
- **Auth / data:** None.
- **Expected:** HTTP `200`.
- **Assertions in suite:** Response status is `200`.
- **Coverage note:** The script does not assert the health response body or database-connected flag.

API-002 — CORS allows configured Vercel origin

- **Request:** `GET /api/health` with `Origin: https://campus-blood.vercel.app`
- **Auth / data:** None.
- **Expected:** Response allows that origin.
- **Assertions in suite:** `Access-Control-Allow-Origin` exactly matches the request origin.

API-003 — CORS allows configured Render origin

- **Request:** `GET /api/health` with `Origin: https://blood-api.onrender.com`
- **Auth / data:** None.
- **Expected:** Response allows that origin.
- **Assertions in suite:** `Access-Control-Allow-Origin` exactly matches the request origin.

API-004 — Seeded donor can log in

- **Request:** `POST /api/auth/login`
- **Auth / data:** Seeded donor account from the local test setup.
- **Expected:** HTTP `200` and a JWT token.
- **Assertions in suite:** Status is `200`; response contains `token`.

API-005 — Seeded coordinator can log in

- **Request:** `POST /api/auth/login`
- **Auth / data:** Seeded coordinator account from the local test setup.
- **Expected:** HTTP `200` and a JWT token.
- **Assertions in suite:** Status is `200`; response contains `token`.

API-006 — Seeded staff account can log in

- **Request:** `POST /api/auth/login`
- **Auth / data:** Seeded staff account from the local test setup.
- **Expected:** HTTP `200` and a JWT token.
- **Assertions in suite:** Status is `200`; response contains `token`.

API-007 — Donor cannot create a drive

- **Request:** `POST /api/drives`
- **Auth / data:** Authenticated donor; test body includes a drive title.

- **Expected:** HTTP `403`.
- **Assertions in suite:** Donor role is forbidden from creating a drive.

API-008 — Coordinator creates a drive and generated slots

- **Request:** `POST /api/drives`
- **Auth / data:** Coordinator token; synthetic two-hour drive, 30-minute duration, capacity one.
- **Expected:** HTTP `201`; four generated slots.
- **Assertions in suite:** Response status is `201`; `drive.slots.length` is `4`.

API-009 — Competing bookings do not overfill a slot

- **Request:** Two simultaneous `POST /api/appointments` requests against the same capacity-one slot.
- **Auth / data:** Two distinct donor tokens; the drive/slot created by API-008.
- **Expected:** Exactly one booking succeeds with HTTP `201`; the losing request returns HTTP `409 Conflict`.
- **Assertions in suite:** The two statuses include both `201` and `409`.
- **Concurrency requirement:** The database transaction/row lock must preserve capacity under concurrent requests; both calls must not succeed.

API-010 — Eligible donor passes pre-screening

- **Request:** `POST /api/appointments/pre-screen`
- **Auth / data:** No authentication required; synthetic answers represent an eligible adult donor.
- **Expected:** HTTP `200`; `eligible` is `true`.
- **Assertions in suite:** The body contains `eligible === true`.

API-011 — Underage and underweight donor fails pre-screening

- **Request:** `POST /api/appointments/pre-screen`
- **Auth / data:** No authentication required; synthetic age 15 and weight 45 kg.
- **Expected:** HTTP `200`; `eligible` is `false`; at least two reasons are returned.
- **Assertions in suite:** `eligible === false` and `reasons.length >= 2`.

API-012 — Staff checks in the successful booking

- **Request:** `PUT /api/staff/appointments/{appointmentId}/checkin`
- **Auth / data:** Staff token; appointment created by the winning API-009 request.
- **Expected:** HTTP `200`; appointment status becomes `CHECKED_IN`.
- **Assertions in suite:** Status is `200`; response appointment status is `CHECKED_IN`.

API-013 — Staff records a completed donation

- **Request:** `POST /api/staff/appointments/{appointmentId}/complete`
- **Auth / data:** Staff token; checked-in appointment; synthetic one-unit outcome.
- **Expected:** HTTP `200`; donation record has `units_collected` equal to `1`.
- **Assertions in suite:** Status is `200`; response record contains one unit.

API-014 — Drive analytics reflect the recorded donation

- **Request:** `GET /api/analytics/drives/{driveId}`
- **Auth / data:** Coordinator token; drive and donation from preceding cases.
- **Expected:** HTTP `200`; `analytics.metrics.total_units_collected` equals `1`.
- **Assertions in suite:** Status is `200`; collected-unit metric equals `1`.

API-015 — CSV export returns report content

- **Request:** `GET /api/analytics/drives/{driveId}/export/csv`
- **Auth / data:** Coordinator token; drive with the completed donation.
- **Expected:** HTTP `200`; CSV text contains the report title.
- **Assertions in suite:** Status is `200`; body is a string containing `CS Blood Donation Drive Coordinator`.
- **Coverage note:** The script does not validate CSV headers, escaping, download headers, or row-level values.

API-016 — Coordinator broadcasts a donor alert

- **Request:** `POST /api/notifications/broadcast`
- **Auth / data:** Coordinator token; synthetic title, message, and target blood groups.
- **Expected:** HTTP `200`; `sent_count` is at least one in the seeded test environment.
- **Assertions in suite:** Status is `200`; `sent_count >= 1`.

4. Recommended test cases not currently asserted by the integration script

Authentication and profile

API-101 — Registration accepts valid synthetic user. `POST /api/auth/register`, no token; expect `201`, token and safe user object, with no `password_hash` in the response.

API-102 — Registration rejects missing required fields. `POST /api/auth/register` with one or more of `full_name`, `email`, or `password` omitted; expect `400`.

API-103 — Registration rejects an existing email. Register the same synthetic email twice; expect the second request to return `400`.

API-104 — Login rejects missing or invalid credentials. `POST /api/auth/login`; expect `400` for missing email/password and `401` for invalid credentials.

API-105 — Authenticated user can read and update own profile. `GET /api/auth/me` then `PUT /api/auth/me` with a new `full_name`; expect `200` and safe user data without a password hash. Also verify missing/invalid bearer tokens are rejected by the authentication middleware.

API-106 — Logout returns success. `POST /api/auth/logout`; expect `200`. Verify client-side token disposal separately because the endpoint response alone does not revoke a stateless JWT.

Drive management

API-107 — Public drive browsing works. `GET /api/drives` and `GET /api/drives/{id}`; expect `200` for existing data and `404` for an unknown drive ID.

API-108 — Drive creation validates required fields. `POST /api/drives` as coordinator/admin with required fields omitted; expect `400`.

API-109 — Drive update is limited to organizer/admin. `PUT /api/drives/{id}`; verify coordinator owner and admin can update, a different coordinator gets `403`, and a missing drive returns `404`.

API-110 — Drive deletion is limited to organizer/admin. `DELETE /api/drives/{id}`; verify owner/admin succeeds with `200`, another coordinator gets `403`, and a missing drive returns `404`.

Donor appointments and capacity

API-111 — Booking requires drive and slot IDs. `POST /api/appointments` as authenticated donor with either identifier omitted; expect `400`.

API-112 — Booking requires authentication. `POST /api/appointments` without a bearer token; expect rejection by authentication middleware.

API-113 — Donor lists personal appointments only. `GET /api/appointments/my-appointments`; verify authenticated donor receives their own appointments and not another donor's records.

API-114 — Cancellation releases slot capacity. `PUT /api/appointments/{id}/cancel`; verify appointment is cancelled and its slot can accept a subsequent booking. Verify a donor cannot cancel another donor's appointment.

API-115 — Rescheduling validates target slot and ownership. `PUT /api/appointments/{id}/reschedule` with `new_slot_id`; expect `400` when omitted, and verify successful reschedule updates capacity on both slots atomically.

API-116 — ICS invite belongs to the authenticated donor. `GET /api/appointments/{id}/ics`; verify valid calendar content for an owned appointment and `404` for an appointment not in the user's appointment list.

API-117 — Full slot rejects additional booking. After filling a slot, attempt another booking and verify conflict response plus unchanged capacity. The automated concurrency test covers a race; this sequential boundary case is additional coverage.

Pre-screening, staff operations, and reporting

API-118 — Pre-screening rejects each disqualifying condition. Exercise age, weight, wellness, recent tattoos, antibiotics, pregnancy, and the 56-day cooldown independently; verify `eligible` and relevant reason(s).

API-119 — Staff queue requires an authorized role. `GET /api/staff/drives/{drive_id}/queue`; verify STAFF/COORDINATOR/ADMIN access, donor denial, and `404` for an unknown drive.

API-120 — Walk-in validation and registration. `POST /api/staff/drives/{drive_id}/walkin`; expect `400` when name/email are missing and `201` for valid synthetic input; check returned appointment and queue state.

API-121 — Queue status validates supported values. `PUT /api/staff/appointments/{id}/status`; expect `400` for an unsupported status and verify a supported status updates the appointment.

API-122 — Donation outcome handles deferral and invalid outcomes. `POST /api/staff/appointments/{id}/complete`; verify `DEFERRED` records a reason and invalid outcomes return `400`. For completed donations, verify the donor's `last_donation_date` is updated as required.

API-123 — Analytics rejects unauthorized roles and unknown drives. `GET /api/analytics/drives/{id}`; verify donor denial and `404` for an unknown drive.

API-124 — CSV download is structurally valid. `GET /api/analytics/drives/{id}/export/csv`; verify `Content-Type`, `Content-Disposition`, headers, escaping of commas/quotes/newlines, and that expected report values correspond to the drive.

Notifications and administration

API-125 — Users can access and mark their own notifications. `GET /api/notifications`, `PUT /api/notifications/{id}/read`, and `PUT /api/notifications/read-all`; verify unread counts update and one user cannot change another user's notification.

API-126 — Broadcast validates payload and role. `POST /api/notifications/broadcast`; missing title/message should return `400`; donors and staff should be denied; coordinator/admin should succeed.

API-127 — Admin user listing and role update are protected. `GET /api/admin/users`, `PUT /api/admin/users/{id}/role`; verify non-admin denial, valid role update with `200`, invalid role with `400`, unknown user with `404`, and no password hashes in output.

API-128 — Admin audit log access is protected. `GET /api/admin/audit-logs`; verify admin access, non-admin denial, and that expected sensitive operations generate audit entries.

5. Route inventory

Area	Method and path
Health	`GET /api/health`
Auth	`POST /api/auth/register`, `POST /api/auth/login`, `POST /api/auth/logout`, `GET /api/auth/me`, `PUT /api/auth/me`
Drives	`GET /api/drives`, `GET /api/drives/{id}`, `POST /api/drives`, `PUT /api/drives/{id}`, `DELETE /api/drives/{id}`
Appointments	`POST /api/appointments/pre-screen`, `POST /api/appointments`, `GET /api/appointments/my-appointments`, `PUT /api/appointments/{id}/cancel`, `PUT /api/appointments/{id}/reschedule`, `GET /api/appointments/{id}/ics`
Staff	`GET /api/staff/drives/{drive_id}/queue`, `POST /api/staff/drives/{drive_id}/walkin`, `PUT /api/staff/appointments/{id}/checkin`, `PUT /api/staff/appointments/{id}/status`, `POST /api/staff/appointments/{id}/complete`
Analytics	`GET /api/analytics/overview`, `GET /api/analytics/drives/{id}`, `GET /api/analytics/drives/{id}/export/csv`
Notifications	`GET /api/notifications`, `PUT /api/notifications/{id}/read`, `PUT /api/notifications/read-all`, `POST /api/notifications/broadcast`
Admin	`GET /api/admin/users`, `PUT /api/admin/users/{id}/role`, `GET /api/admin/audit-logs`

6. Important test-data and safety notes

- Use a disposable test database and synthetic donors only.

- Keep the two actors in the concurrency test distinct; same-user duplicate rules are not a substitute for testing two simultaneous transactions.
- Assert persisted slot capacity and appointment count after concurrent booking, in addition to HTTP statuses, when adding deeper integration coverage.
- Store test credentials in local environment configuration, not in this document.
- Pre-screening responses may be sensitive. Keep fixtures synthetic and avoid logging or retaining full health-answer payloads unnecessarily.
- Test server/database failures and error response formatting without exposing stack traces, SQL details, JWT secrets, or `password_hash`.